



Lighting

04 Lighting

램버트 라이팅

게임 프로그래머 | 이창진

| 목차

Chapter 1. 빛의 정의

Chapter 2. 빛의 종류

Chapter 3. 법선벡터

Chapter 4. 램버트 코사인 정리

Chapter 5. 셰이더 코드 주의할 점

빛이란?

정의

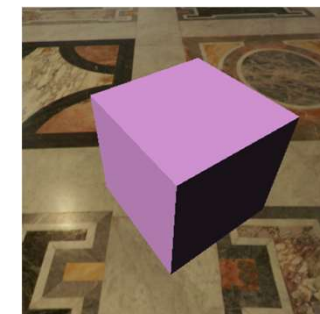
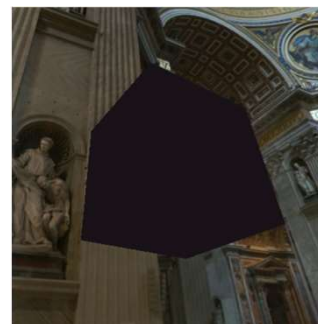
물질이나 사물이 나타내는 색입니다.

조금 더 정확히는 여러 광선들 중에 인간의 눈이 관측 가능한 가시광선이 눈에 들어온 모든 총량입니다. 반사, 굴절, 간섭, 회절 등의 모든 효과가 종합된 빛을 우리 눈이 감지합니다.

요약하자면 빛은 색입니다.

PPT에서 다루는 빛

월드 노말과 라이트 방향의 반대를 내적하여 빛을 연산하는 램버트 라이트를 사용합니다.



Directional Light



Point Light



Spot Light



RectLight



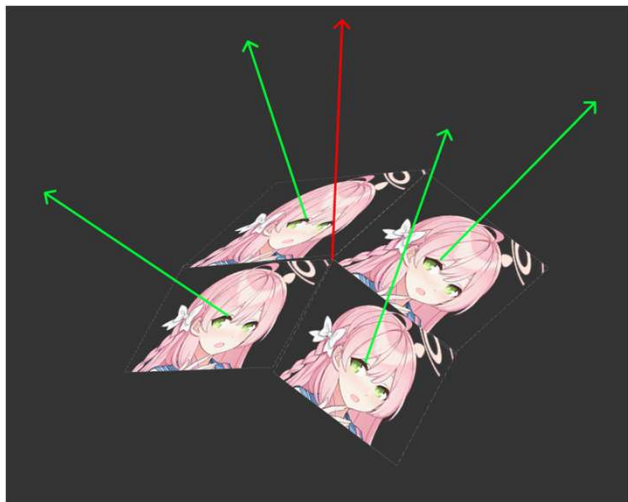
법선 벡터



정의

법선 벡터는 물체 표면의 방향을 나타내는데 사용됩니다.

표면에 수직인 단위 벡터입니다. 이 벡터는 라이팅 계산에 주로 사용됩니다. 우리가 알고 있는 텍스처 맵핑에 사용됩니다.



계산

어떤 꼭짓점에 인접한 면들의 법선이 n_0, n_1, n_2, n_3 라고 할때, 해당 꼭짓점들의 평균 법선 벡터는 다음과 같습니다

$$\vec{n}_{avg} = \frac{\vec{n}_0 + \vec{n}_1 + \vec{n}_2 + \vec{n}_3}{\|\vec{n}_0 + \vec{n}_1 + \vec{n}_2 + \vec{n}_3\|}$$

곡면에 대해서 생각해봅시다. 다음과 같이 함수의 영집합으로 정의되어 있다고 해봅시다.

$$f(x, y, z) \equiv 0$$

곡면 위의 한 점 (x, y, z) 에서의 법선 벡터(정규화 전)는 함수 f 의 **그래디언트(gradient)**로 주어집니다:

$$\vec{n} = \nabla f(x, y, z) = \frac{\partial f}{\partial x} \vec{i} + \frac{\partial f}{\partial y} \vec{j} + \frac{\partial f}{\partial z} \vec{k}$$

최종적으로 단위 법선은 다음과 같습니다

$$\hat{n} = \frac{\vec{n}}{\|\vec{n}\|}$$

법선 벡터 – 비균등 스케일 문제

설명

법선 벡터는 비균등 스케일에서 값이 달라집니다.

접선 벡터 $t = (0, 1, 0)$ 이라고

법선 벡터 $n = (0, 0, 1)$ 이라 해봅시다

그리고 변환 행렬은 다음과 같다고 해봅시다

$$M = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 3 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

변환 후 벡터들은 다음과 같습니다

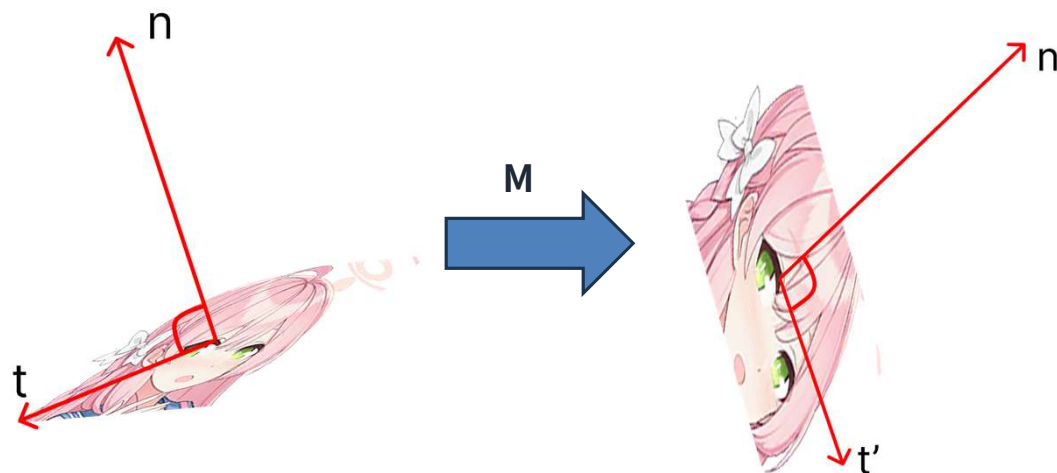
$$t' = Mt = (0, 3, 1)$$

$$n' = Mn = (0, 1, 1)$$

이 둘의 내적은 다음과 같습니다

$$t' \cdot n' = 0 \times 0 + 3 \times 1 + 1 \times 1 = 4$$

즉 수직성이 깨집니다.



해결하는 방법

법선 벡터 n 은 접선 벡터 t 와 수직하므로 항상

$$(n^T)t = 0$$

변환 후 법선 벡터 n' 과 접선 벡터 $t' = Mt$ 도 수직이어야 하므로,

$$(n')^T t' = (n')^T (Mt) = (n'^T M)t = 0$$

이 모양을 잘 보면

$$n'^T = n^T M$$

양변에 전치를 하고 정리하면

$$n' = (M^T)^{-1} n = (M^{-1})^T n$$

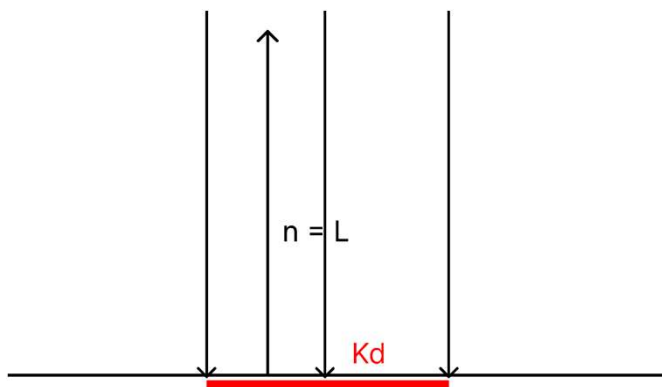
이 됩니다.

람버트 코사인 정리



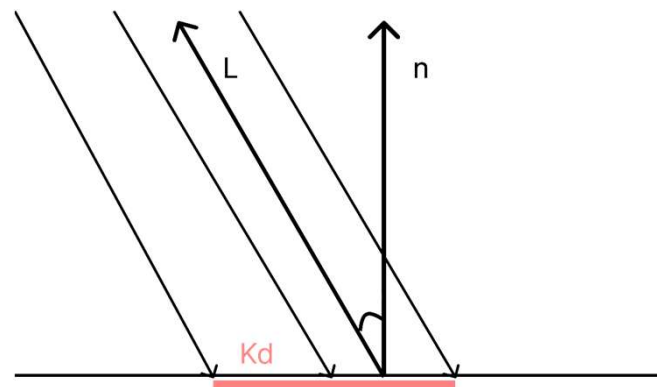
Theta = 90

법선 벡터와 빛이 이루는 각도가 90도 일때



0 < Theta < 90

법선 벡터와 빛이 이루는 각도가 0도 이상 90도 이하 일때



$$kd = \max(L \cdot n, 0)$$

L: 빛의 입사 방향(단위 벡터)
n: 표면의 단위 법선 벡터
kd: 난반사 세기 계수 (0 이상)

쉐이더 코드 주의할 점



BasicPS

틀림!

```
#include <shared.fhx>
```

```
// 픽셀 셰이더(쉐이더/셰이더)
float4 main(VertexOut pln) : SV_TARGET
{
    float3 baseColor = float3(1.0, 1.0, 1.0);
    pln.normalW = normalize(pln.normalW);
    float saturatedDir = saturate(dot(g_DirLight.direction.xyz, pln.normalW));

    float3 lit = g_DirLight.color.rgb + g_DirLight.color.rgb * saturatedDir;
    float3 rgb = baseColor * lit;

    return float4(rgb, 1.0f);
}
```

전역으로 설정되어 있는 빛의 방향과 면의 노말을 내정해
saturate하면 될 것이라고 생각함. -> 틀림

BasicPS

```
#include "10_shared.fhx"
```

```
// 픽셀 셰이더(쉐이더/셰이더)
float4 main(VertexOut pln) : SV_Target
{
    float3 baseColor = float3(1.0, 0.7, 1.0);
    float3 lightVec = -g_DirLight.direction;
    float diffuseFactor = saturate(dot(lightVec, pln.normalW));

    float3 lit = 0.1 * g_DirLight.color.rgb + diffuseFactor * g_DirLight.color.rgb;
    float3 litColor = baseColor * lit;

    return float4(litColor, 1.0f);
}
```

빛은 반대방향이라 음수를 곱해서 내적해야함. 그리고
지금만 해당 면에 대한 빛이지만, 원래는 eyeVector를
만들어서 눈에 들어오는 총 량을 구해야합니다

셰이더 코드 주의할 점



BasicVS

틀림!

```
#include <shared.fhx>
// 정점 셰이더.
VertexOut main(VertexIn vIn)
{
    VertexOut vOut;

    float4 posW = mul(float4(vIn.posL, 1.0f), g_World);
    vOut.posW = posW.xyz;
    vOut.normalW = mul(float4(vIn.normalL, 0.0f), g_WorldInvTranspose).xyz;
    float4 posH = mul(posW, g_View);
    vOut.posH = mul(posH, g_Proj);
    vOut.color = vIn.color;

    return vOut;
}
```

법선이 월드의 T행렬이라고 생각하고 `g_WorldInvTranspose`를 구하고
사용하려고 했습니다. 즉 법선은 위치와 다르게 변환된다 -> 역전치로
변환해야한다 로 이해하고 코딩했습니다. 그래서 `w=0`으로 이동 성분을
제거한 뒤에 `g_WorldInvTranspose`를 곱했습니다.

BasicVS

```
#include "10_shared.fhx"
// 정점 셰이더
VertexOut main(VertexIn vIn)
{
    VertexOut vOut;

    float4 posW = mul(float4(vIn.posL, 1.0f), g_World);
    vOut.posH = mul(posW, g_View);
    vOut.posH = mul(vOut.posH, g_Proj);
    vOut.posW = posW.xyz;

    vOut.normalW = normalize(mul(vIn.normalL, (float3x3)
    g_WorldInvTranspose));
    vOut.color = vIn.color;

    return vOut;
}
```

다만 지금 프로젝트가 가정하는 월드 = 회전 + 이동 과 설정 방식이 일치하지 않았고
Normalize를 여기서 진행하지 않고 PS에서 `normaliz`해서 사용하려고 하다보니
카메라의 회전을 빛의 방향이 따라오게 되었습니다. 이를 해결하기위해 VS에서는
3*3 행렬로 `g_WorldInvTranspose`를 곱한 후에 Normalize 했습니다.

감사합니다.